

- 1 Проект explorer: Полная заметка к защите
 - 1.1 Как пользоваться этим документом
 - 1.2 1. Цель проекта и бизнес-логика
 - 1.2.1 1.1. Что делает система
 - 1.2.2 1.2. Главные требования, которые фактически реализованы
 - 1.3 2. Технологический стек
 - 1.3.1 2.1. Язык и платформа
 - 1.3.2 2.2. Web слой
 - 1.3.3 2.3. Persistence
 - 1.3.4 2.4. Безопасность
 - 1.4 3. Архитектура проекта
 - 1.5 3.1. Слои
 - 1.5.1 3.2. Почему это хорошая учебная архитектура
 - 1.5.2 3.3. Диаграмма взаимодействия (логин)
 - 1.6 4. Детальный разбор по классам
 - 1.6.1 4.1. AuthService
 - 1.6.2 4.2. UserRepository (актуально: Hibernate)
 - 1.6.3 4.3. HibernateUtil
 - 1.6.4 4.4. UserEntity
 - 1.6.5 4.5. PasswordService
 - 1.6.6 4.6. Сервлеты
 - 1.6.7 4.7. JSP
 - 1.7 5. HTTP/Servlet сценарии по шагам (очень подробно)
 - 1.7.1 5.1. Регистрация
 - 1.7.2 5.2. Логин
 - 1.7.3 5.3. Просмотр файлов
 - 1.7.4 5.4. Скачивание
 - 1.8 6. БД и ORM: что именно происходит
 - 1.8.1 6.1. Физическая таблица
 - 1.8.2 6.2. Что такое SessionFactory/Session на пальцах
 - 1.8.3 6.3. Почему HQL, а не raw SQL
 - 1.8.4 6.4. Ограничения текущей реализации
 - 1.9 7. Эволюция версий: auth -> added MySQL DB -> текущая Hibernate-версия
 - 1.10 7.1. Стадия added auth (415a4ea)
 - 1.10.1 7.2. Стадия added MySQL DB (d850814) — что важно знать для защиты
 - 1.10.2 7.3. Текущая версия (working tree) — переход JDBC -> Hibernate
 - 1.10.3 7.4. Таблица «что было / что стало»
 - 1.10.4 7.5. Что важно подчеркнуть на защите по сравнению версий
 - 1.11 8. Безопасность: что уже сделано и что можно улучшить
 - 1.11.1 8.1. Уже сделано хорошо
 - 1.11.2 8.2. Риски, которые остались
 - 1.11.3 8.3. Что можно предложить как roadmap
 - 1.12 9. Производительность и стабильность
 - 1.12.1 9.1. Где потенциальные bottlenecks
 - 1.12.2 9.2. Почему для учебного проекта это ок
 - 1.13 10. Полный запуск проекта (локальный процесс, без Docker)
 - 1.14 10.1. Требования окружения
 - 1.14.1 10.2. Сборка
 - 1.14.2 10.3. Деплой WAR в Tomcat

- 1.14.3 10.4. JVM параметры для Tomcat
- 1.14.4 10.5. Создание БД
- 1.14.5 10.6. URL
- 1.14.6 10.7. DBeaver
- 1.15 11. Что показать на живой демонстрации
- 1.16 12. Частые вопросы от преподавателя: готовые ответы
 - 1.16.1 12.1. Базовые
 - 1.16.2 12.2. По Hibernate/JPA
 - 1.16.3 12.3. По безопасности
 - 1.16.4 12.4. По эволюции проекта
- 1.17 13. Продвинутые вопросы (уровень «копаем глубже»)
 - 1.17.1 13.1. Почему repository still throws SQLException, если внутри ORM?
 - 1.17.2 13.2. Зачем в UserRepository.authenticate используется setMaxResults(1)?
 - 1.17.3 13.3. Почему не используете getSingleResult()?
 - 1.17.4 13.4. Что произойдет при конкурентной регистрации одинакового логина?
 - 1.17.5 13.5. Что если в path передать URL-encoded строку с ..?
 - 1.17.6 13.6. Где слабое место в session handling?
- 1.18 14. Разбор «по файлам» — краткий индекс
- 1.19 15. Известные технические замечания (честно, как на хорошей защите)
- 1.20 16. Короткий «спич» на 60–90 секунд
- 1.21 17. Приложение А: команды для диагностики
 - 1.21.1 17.1. Проверка MySQL
 - 1.21.2 17.2. Проверка Tomcat и порта
 - 1.21.3 17.3. Быстрый smoke test
- 1.22 18. Приложение В: каркас устных ответов «почему так»
- 1.23 19. Приложение С: контрольный список перед защитой
- 1.24 20. Итог
- 1.25 21. Servlet контейнер: глубокий разбор lifecycle
 - 1.25.1 21.1. Что делает Tomcat на старте
 - 1.25.2 21.2. Forward vs Redirect (частый вопрос)
 - 1.25.3 21.3. Session lifecycle в этом проекте
 - 1.25.4 21.4. Потокбезопасность
- 1.26 22. Hibernate internals: что могут спросить глубже
 - 1.26.1 22.1. Entity states
 - 1.26.2 22.2. Flush и Commit
 - 1.26.3 22.3. Почему SessionFactory static final
 - 1.26.4 22.4. Почему Session на операцию
 - 1.26.5 22.5. Что делает hbm2ddl.auto=update
 - 1.26.6 22.6. Почему в логе предупреждение про dialect
- 1.27 23. Полная матрица эволюции (3 стадии)
- 1.28 24. Разбор ключевых строк по файлам (для «покажи в коде»)
 - 1.28.1 24.1. AuthService
 - 1.28.2 24.2. UserRepository
 - 1.28.3 24.3. HibernateUtil
 - 1.28.4 24.4. Servlet-слой
- 1.29 25. Каверзные вопросы и как на них отвечать
 - 1.29.1 25.1. «Почему не оставили JDBC, если он проще и прозрачнее?»
 - 1.29.2 25.2. «Что будет, если Hibernate не сможет поднять SessionFactory?»
 - 1.29.3 25.3. «Почему не используете @Transactional?»

- 1.29.4 25.4. «Не слишком ли рано вы делаете `ensureDatabase()` на `login/register?`»
- 1.29.5 25.5. «Почему `login/email` уникальны и что будет при конфликте?»
- 1.29.6 25.6. «Возможна ли `SQL/HQL injection?`»
- 1.29.7 25.7. «Почему не сделали слой DAO интерфейс + `impl?`»
- 1.29.8 25.8. «Как проверяется принадлежность файла пользователю?»
- 1.29.9 25.9. «Почему в проекте есть `Main.java?`»
- 1.29.10 25.10. «Почему в проекте остался `DbConnectionFactory?`»
- 1.30 26. Что лучше улучшить после защиты (если спросят `roadmap`)
- 1.31 27. Мини-шпаргалка (одной страницей)
- 1.32 28. Приложение D: разница по файлам (конкретно)
 - 1.32.1 28.1. Что добавил `added MySQL DB`
 - 1.32.2 28.2. Что добавила текущая `Hibernate-миграция (working tree)`
 - 1.32.3 28.3. Что внешне не поменялось
- 1.33 29. Приложение E: сверхкраткий рассказ по `commit-истории`
- 1.34 30. Финальный вывод для защиты

1 Проект `explorer`: Полная заметка к защите

Версия документа: 2026-04-28

Проект: Java web-приложение (`Servlet/JSP`) для файлового менеджера с регистрацией/логином и хранением пользователей в `MySQL` через `Hibernate`.

Исторические точки, которые учитываются в защите:

- `3602ddf` — `init`
- `415a4ea` — `added auth`
- `d850814` — `added MySQL DB` (последний закоммиченный `commit` в `main`)
- Актуальное рабочее состояние (`working tree` на дату документа) — миграция `persistence-слоя` с `JDBC` на `Hibernate` (еще не закоммичено)

1.1 Как пользоваться этим документом

1. Перед защитой прочитай разделы 1–4 и 8 (это «база»).
 2. Перед демонстрацией открой раздел 10 (чеклист запуска).
 3. Для вопросов преподавателя используй раздел 13 (Q&A).
 4. Для вопросов по изменениям «прошлый `commit` vs текущая версия» используй раздел 7.
-

1.2 1. Цель проекта и бизнес-логика

1.2.1 1.1. Что делает система

Приложение реализует защищенный файловый менеджер с web-интерфейсом:

- пользователь может зарегистрироваться (/register),
- войти (/login),
- после входа увидеть список файлов своего домашнего каталога (/files),
- скачать файл (/download),
- выйти (/logout).

Ключевая бизнес-идея: каждый пользователь имеет собственный изолированный каталог в ~/filemanager/homes/<login>.

1.2.2 1.2. Главные требования, которые фактически реализованы

- Аутентификация и авторизация на уровне HTTP-сессии (JSESSIONID, session attribute userLogin).
 - Разделение пользовательских файлов по home-каталогу.
 - Защита от path traversal (проверка isInsideHome).
 - Хранение учеток в MySQL.
 - Хранение пароля в виде bcrypt-хеша, а не в открытом виде.
 - Миграция persistence-слоя с JDBC на Hibernate (актуальное состояние).
-

1.3 2. Технологический стек

1.3.1 2.1. Язык и платформа

- Java 17
- Maven

1.3.2 2.2. Web слой

- Jakarta Servlet API 6.0
- JSP + scriptlets (без Spring MVC)
- Tomcat 10.1+

1.3.3 2.3. Persistence

- MySQL 8/9 (локальный процесс)
- В commit added MySQL DB: JDBC + mysql-connector-j
- В актуальной версии: Hibernate ORM 6.4 + Jakarta Persistence API 3.1

1.3.4 2.4. Безопасность

- org.mindrot:jbcrypt для хеширования паролей
-

1.4 3. Архитектура проекта

1.5 3.1. Слои

Текущая архитектура (актуальная версия):

- Presentation: HomeServlet, LoginServlet, RegisterServlet, FilesServlet, DownloadServlet, LogoutServlet + JSP
- Application/Domain service: AuthService
- Persistence service: UserRepository
- ORM bootstrap: HibernateUtil
- Entity: UserEntity
- Crypto utility: PasswordService

Структура потока:

HTTP Request -> Servlet -> AuthService -> UserRepository ->
Hibernate Session -> MySQL

1.5.1 3.2. Почему это хорошая учебная архитектура

- Простая (без DI-фреймворка, без лишних абстракций).
- Четкое разделение responsibilities.
- Легко показывать evolution проекта по коммитам.
- Можно объяснить и на базовом уровне, и на уровне internals (session/transaction).

1.5.2 3.3. Диаграмма взаимодействия (логин)

Browser

-> POST /login (login,password)

Tomcat

-> LoginServlet#doPost

-> AuthService.authenticate

-> UserRepository.authenticate

-> HibernateUtil.openSession

-> HQL query by login

-> compare bcrypt hash

<- boolean authenticated

-> if true: HttpSession#setAttribute("userLogin", login)

-> 302 /files

Browser -> GET /files

1.6 4. Детальный разбор по классам

1.6.1 4.1. AuthService

Файл: `src/main/java/org/example/AuthService.java`

Ключевые задачи:

- Нормализация и валидация входных данных.
- Координация `register/authenticate` сценариев.
- Создание `home`-каталогов.
- Гейт инициализации базы (`ensureDatabase`).

Ключевые фрагменты:

- `register(...)` — строки 20–55:
 - делает `ensureStorage()` и `ensureDatabase()`;
 - валидирует логин/пароль/email;
 - вызывает `PasswordService.hash(...)`;
 - делегирует сохранение в `UserRepository.createUser(...)`;
 - при успехе создает `home`-папку.
- `authenticate(...)` — строки 57–81:
 - валидирует вход;
 - вызывает `UserRepository.authenticate(...)`;
 - при успехе гарантирует существование `home`-папки.
- `resolveInsideHome(...)` + `isInsideHome(...)` — строки 92–116:
 - защищает от перехода за пределы домашней папки пользователя.

Аргументы на защите:

- Это `application service`, где централизованы правила домена.
- Сервлеты остаются `thin-controller`.
- `Storage` и `DB` инициализация вынесены из `presentation`-слоя.

1.6.2 4.2. UserRepository (актуально: Hibernate)

Файл: `src/main/java/org/example/UserRepository.java`

Ключевые задачи:

- Инициализация ORM-schema (`ensureSchema`).
- Запись пользователя (`createUser`) в транзакции.
- Поиск пользователя для аутентификации (`authenticate`).

Что важно:

- `createUser`:
 - открывает `Session` (`try-with-resources`),
 - открывает `Transaction`,
 - `persist(new UserEntity(...))`,
 - `commit`.
- На исключении делает `rollback` (`rollbackQuietly`).
- Конфликт `unique` обрабатывается как «пользователь уже существует».

Аргументы на защите:

- Репозиторий не знает про HTTP/Servlet.
- Весь SQL/ORM доступ изолирован в одном слое.
- Транзакция на запись корректно закрывается.

1.6.3 4.3. HibernateUtil

Файл: `src/main/java/org/example/HibernateUtil.java`

Роль:

- Singleton-style bootstrap `SessionFactory`.
- Конфигурация подключения и `Hibernate properties`.

Почему так:

- `SessionFactory` тяжелый объект, создается один раз.
- `Session` дешевле и открывается на каждую операцию.

Ключевые properties:

- `hibernate.connection.url/user/pass`
- `hibernate.hbm2ddl.auto` (по умолчанию `update`)
- `hibernate.show_sql`
- `hibernate.format_sql`

Плюс:

- Поддерживает переопределение через `system properties/env vars` (`explorer.*`, `EXPLORER_*`).

1.6.4 4.4. UserEntity

Файл: `src/main/java/org/example/UserEntity.java`

Маппинг:

- `@Entity`
- `@Table(name = "users")`
- `id` — `@Id + @GeneratedValue(IDENTITY)`
- `login, email` — `unique, nullable=false`
- `password` — `nullable=false`

Аргумент на защите:

- Это полноценный ORM mapping на существующую логическую модель таблицы `users`.

1.6.5 4.5. PasswordService

Файл: `src/main/java/org/example/PasswordService.java`

Что делает:

- `hash(raw)` — `bcrypt(gensalt(12))`.
- `matches(raw, stored)` — проверка `bcrypt` hash.
- `isBcryptHash(...)` — guard для формата.

Аргумент на защите:

- Пароли не хранятся plaintext (улучшение относительно стадии `added auth`).

1.6.6 4.6. Сервлеты

1.6.6.1 HomeServlet

- Маршрут `""`, `"/`.
- Редиректит на `/login` или `/files` в зависимости от `session`.

1.6.6.2 LoginServlet

- GET отдаёт форму логина.
- POST вызывает `AuthService.authenticate`.
- При успехе создаёт `session` и ставит `userLogin`.

1.6.6.3 RegisterServlet

- GET отдаёт форму регистрации.
- POST вызывает `AuthService.register`.
- При успехе автологин через `session`.

1.6.6.4 FilesServlet

- Проверяет `session`.
- Разрешает только пути внутри `home`.
- Сортирует файлы.
- Передаёт данные в JSP.

1.6.6.5 DownloadServlet

- Проверяет `session`.
- Разрешает скачивание только из `home`.
- Выставляет корректные HTTP headers для `download`.

1.6.6.6 LogoutServlet

- POST: `session.invalidate()` -> `redirect /login`.

1.6.7 4.7. JSP

- `login.jsp`, `register.jsp`, `files.jsp`.
- Используются `server-side scriptlets`.

- Для учебного проекта допустимо, хотя в production обычно переходят на JSTL/EL/templating.
-

1.7 5. HTTP/Servlet сценарии по шагам (очень подробно)

1.7.1 5.1. Регистрация

1. Browser: GET /register.
2. RegisterServlet#doGet проверяет session.
3. Если не залогинен — forward register.jsp.
4. Browser: POST /register с form data.
5. RegisterServlet#doPost вызывает AuthService.register.
6. AuthService.register:
 - ensureStorage;
 - ensureDatabase -> UserRepository.ensureSchema;
 - validate input;
 - hash password;
 - UserRepository.createUser.
7. UserRepository.createUser:
 - open Session;
 - begin Transaction;
 - persist entity;
 - commit;
 - close Session.
8. Возврат в сервлет:
 - успех -> создать session -> userLogin -> redirect /files;
 - ошибка -> request.setAttribute(error) -> forward на форму.

1.7.2 5.2. Логин

1. Browser: POST /login.
2. LoginServlet#doPost -> AuthService.authenticate.
3. AuthService.authenticate -> UserRepository.authenticate.
4. Репозиторий делает HQL query by login.
5. Сравнение хеша через PasswordService.matches.
6. При успехе сервлет записывает userLogin в session и редиректит на /files.

1.7.3 5.3. Просмотр файлов

1. Browser: GET /files?path=....
2. FilesServlet#doGet:
 - читает session.userLogin;
 - запрещает доступ без сессии;
 - получает home;
 - вызывает resolveInsideHome(login, path);
 - если путь вне home => 403;
 - если неверная директория => 400;

- сортирует и передаёт в JSP.

1.7.4 5.4. Скачивание

1. Browser: GET /download?path=....
 2. DownloadServlet:
 - проверка session;
 - проверка path;
 - проверка принадлежности home;
 - проверка file exists;
 - потоковая отдача файла в response output stream.
-

1.8 6. БД и ORM: что именно происходит

1.8.1 6.1. Физическая таблица

Таблица users:

- id BIGINT AUTO_INCREMENT PK
- login VARCHAR(32) UNIQUE NOT NULL
- email VARCHAR(255) UNIQUE NOT NULL
- password VARCHAR(255) NOT NULL

В текущей ORM-версии schema поддерживается через `hibernate.hbm2ddl.auto=update`.

1.8.2 6.2. Что такое SessionFactory/Session на пальцах

- SessionFactory — фабрика с метаданными mapping и настройками; потокобезопасна.
- Session — контекст работы с БД в рамках одной операции; не потокобезопасна.
- На запись: `beginTransaction -> persist -> commit`.

1.8.3 6.3. Почему HQL, а не raw SQL

- HQL работает в терминах сущностей (UserEntity), а не таблиц.
- Удобнее эволюционировать domain model.
- Параметр `:login` исключает SQL injection на этом запросе.

1.8.4 6.4. Ограничения текущей реализации

- `show_sql` и `hbm2ddl` конфигурируются через свойства, но нет отдельного `hibernate.cfg.xml`.
 - Нет pooling/DataSource (Hibernate built-in pool не production-grade).
 - Для учебного проекта это нормально.
-

1.9 7. Эволюция версий: auth -> added MySQL DB -> текущая Hibernate-версия

1.10 7.1. Стадия added auth (415a4ea)

Основная идея:

- регистрация/логин реализованы через файлы `.properties`.
- пароль хранился в plaintext.

Плюсы:

- быстро и просто для MVP.

Минусы:

- нет централизованной БД;
- слабая безопасность паролей;
- сложнее масштабировать.

1.10.1 7.2. Стадия added MySQL DB (d850814) — что важно знать для защиты

Этот commit сделал ключевой переход с файлов на БД.

Изменения:

- `pom.xml`: добавлены `mysql-connector-j`, `bcrypt`.
- Добавлен `DbConnectionFactory`.
- Добавлен `UserRepository` (JDBC SQL).
- Добавлен `PasswordService` (bcrypt).
- `AuthService` переписан:
 - убраны `.properties` users,
 - добавлено `ensureDatabase` + вызовы репозитория,
 - plaintext сравнение заменено на `bcrypt`.

Результат:

- данные пользователей теперь в MySQL,
- пароль хранится безопаснее,
- появилась явная сложность persistence.

Типичные вопросы по этому commit и короткие ответы:

- Почему добавили `ensureSchema`?
Чтобы таблица `users` гарантированно была создана перед `register/login`.
- Почему `SQLIntegrityConstraintViolationException` ловится отдельно?
Это индикатор duplicate login/email из UNIQUE constraints.
- Почему `bcrypt cost = 12`?
Баланс между безопасностью и временем вычисления для учебного проекта.

1.10.2 7.3. Текущая версия (working tree) — переход JDBC -> Hibernate

Состояние после незакоммиченных изменений:

- В pom.xml добавлены hibernate-core и jakarta.persistence-api.
- Новый UserEntity (@Entity).
- Новый HibernateUtil (SessionFactory).
- UserRepository переписан на ORM API.
- AuthService/Servlet/JSP практически не менялись по внешнему контракту.

1.10.3 7.4. Таблица «что было / что стало»

Область	Было в d850814	Стало в текущей версии
Persistence API	JDBC (Connection, PreparedStatement)	Hibernate (Session, Transaction)
Модель данных в Java	SQL-строки напрямую	UserEntity + HQL
Инициализация БД	CREATE TABLE IF NOT EXISTS в UserRepository.ensureSchema	SessionFactory + hbm2ddl=update
CRUD пользователя	INSERT/SELECT руками	persist + HQL query
Конфигурация	DbConnectionFactory	HibernateUtil

1.10.4 7.5. Что важно подчеркнуть на защите по сравнению версий

- Наружный API приложения (URL, формы, session) сохранился.
- Изменился только слой доступа к данным.
- Это пример безопасного refactoring с минимальной ломкой верхних слоев.

1.11 8. Безопасность: что уже сделано и что можно улучшить

1.11.1 8.1. Уже сделано хорошо

- Пароли в bcrypt, а не plaintext.
- Защита от path traversal (canonical path + prefix check).
- Проверка аутентификации перед доступом к /files и /download.
- Ограничение логина regex-ом и длиной.

1.11.2 8.2. Риски, которые остались

- Нет CSRF защиты форм.

- Нет rate-limit на попытки логина.
- Нет forced session rotation при login (технически session создается/переиспользуется стандартно).
- Нет разграничения ролей (только «пользователь»).

1.11.3 8.3. Что можно предложить как roadmap

- CSRF token в формы.
 - Ограничение попыток логина/lockout.
 - Перейти с JSP scriptlets на JSTL/EL.
 - Добавить Filter для единой проверки аутентификации на protected routes.
 - Вынести DB credentials в env/secrets.
-

1.12 9. Производительность и стабильность

1.12.1 9.1. Где потенциальные bottlenecks

- Для каждой auth-операции создается новая ORM session.
- Нет production-grade connection pool.
- На больших директориях листинг в /files может быть дорогим (listFiles + сортировка).

1.12.2 9.2. Почему для учебного проекта это ок

- Нагрузка маленькая.
 - Логика прозрачна и легко объяснима.
 - Упрощает отладку и защиту.
-

1.13 10. Полный запуск проекта (локальный процесс, без Docker)

1.14 10.1. Требования окружения

- Java 17
- Maven
- Tomcat 10.1+
- MySQL

1.14.1 10.2. Сборка

```
mvn -DskipTests clean package
```

1.14.2 10.3. Деплой WAR в Tomcat

WAR: target/explorer-1.0-SNAPSHOT.war

Скопировать в webapps/explorer.war.

1.14.3 10.4. JVM параметры для Tomcat

Передать через CATALINA_OPTS, например:

```
-Dexplorer.db.url=jdbc:mysql://localhost:3306/explorer  
-Dexplorer.db.user=root  
-Dexplorer.db.password=root  
-Dexplorer.hibernate.hbm2ddl=update  
-Dexplorer.hibernate.show_sql=true
```

1.14.4 10.5. Создание БД

```
CREATE DATABASE IF NOT EXISTS explorer CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;  
CREATE USER IF NOT EXISTS 'explorer'@'localhost' IDENTIFIED BY  
'explorer';  
GRANT ALL PRIVILEGES ON explorer.* TO 'explorer'@'localhost';  
FLUSH PRIVILEGES;
```

1.14.5 10.6. URL

- http://localhost:8080/explorer

1.14.6 10.7. DBeaver

- Host: 127.0.0.1
 - Port: 3306
 - DB: explorer
 - User/pass: согласно твоей локальной настройке
 - Если ошибка Public Key Retrieval is not allowed:
 - allowPublicKeyRetrieval=true
 - useSSL=false
-

1.15 11. Что показать на живой демонстрации

Демо-скрипт (3–5 минут):

1. Открыть /explorer -> редирект на логин.
2. Зарегистрировать нового пользователя.
3. Показать, что после регистрации идет вход в /files.
4. Показать в DBeaver, что в users новая строка с bcrypt hash.
5. Выйти (logout), снова войти тем же аккаунтом.

6. Попробовать вручную подставить path вне home и показать 403.

Что проговорить вслух:

- «Сессия хранит только login пользователя, доступ к файлам валидируется на каждый запрос».
 - «Пароль не хранится в открытом виде, только bcrypt».
 - «Persistence слой мигрирован с JDBC на Hibernate без изменения URL/API».
-

1.16 12. Частые вопросы от преподавателя: готовые ответы

1.16.1 12.1. Базовые

Q: Почему выбрали Servlet/JSP, а не Spring?

A: Это учебный модуль по базовой web-механике Java. Servlet/JSP позволяет показать lifecycle, session и маршрутизацию без скрытой магии фреймворка.

Q: Где хранится состояние «кто залогинен»?

A: В HttpSession атрибуте userLogin.

Q: Где хранятся пользователи?

A: В таблице users MySQL.

Q: Пароль хранится как?

A: Bcrypt hash (PasswordService.hash).

1.16.2 12.2. По Hibernate/JPA

Q: В чем разница JPA и Hibernate?

A: JPA — спецификация API, Hibernate — реализация ORM + native API (Session).

Q: Что такое SessionFactory?

A: Потокбезопасная фабрика Session, создается один раз на приложение.

Q: Почему Session не хранится глобально?

A: Session не thread-safe, должна жить коротко в рамках одной операции.

Q: Где транзакция?

A: В UserRepository.createUser: beginTransaction -> persist -> commit.

Q: Почему hbm2ddl=update, а не create?

A: update не дропает схему при каждом старте, безопаснее для разработки.

1.16.3 12.3. По безопасности

Q: Как защищены от path traversal?

A: resolveInsideHome + getCanonicalFile + isInsideHome prefix-check.

Q: SQL injection есть?

A: На auth-запросе нет: параметризованный HQL (: login).

Q: Что еще нужно для production?

A: CSRF, rate-limit, connection pool, централизованный auth filter, audit logs.

1.16.4 12.4. По эволюции проекта

Q: Что сделал commit added MySQL DB?

A: Убрал файловое хранение users, добавил MySQL/JDBC-слой, bcrypt, schema init.

Q: Что изменилось после этого в текущей версии?

A: JDBC слой заменен на Hibernate (UserEntity, HibernateUtil, ORM-переписанный UserRepository).

1.17 13. Продвинутые вопросы (уровень «копаем глубже»)

1.17.1 13.1. Почему repository still throws SQLException, если внутри ORM?

Это сделано для совместимости с существующим контрактом AuthService (минимальные правки верхних слоев). Можно рефакторить на custom exceptions domain/persistence.

1.17.2 13.2. Зачем в UserRepository.authenticate используется setMaxResults(1)?

Чтобы явно ограничить выборку в случае аномалии данных и не тянуть больше одной записи.

1.17.3 13.3. Почему не используете getSingleResult()?

getSingleResult() кидает исключения при NoResultException/NonUniqueResultException; через список проще контролировать happy-path в учебном коде.

1.17.4 13.4. Что произойдет при конкурентной регистрации одинакового логина?

Сработает unique constraint в БД, Hibernate/JDBC драйвер бросит exception, он распознается как duplicate и преобразуется в user-friendly ошибку.

1.17.5 13.5. Что если в path передать URL-encoded строку с ...?

getCanonicalFile нормализует путь, затем isInsideHome отсекает выход за домашнюю директорию.

1.17.6 13.6. Где слабое место в session handling?

Нет отдельного auth-filter и fine-grained route policy; проверка размазана по сервлетам. Для большого проекта лучше Filter/Interceptor.

1.18 14. Разбор «по файлам» — краткий индекс

- pom.xml — зависимости, packaging war.
 - AuthService — бизнес-правила регистрации/логина + home-path guards.
 - UserRepository — persistence API (сейчас Hibernate).
 - HibernateUtil — ORM bootstrap.
 - UserEntity — JPA mapping таблицы users.
 - PasswordService — bcrypt.
 - HomeServlet — root redirect.
 - LoginServlet/RegisterServlet — auth endpoints.
 - FilesServlet — список файлов.
 - DownloadServlet — скачивание.
 - LogoutServlet — logout.
 - WEB-INF/*.jsp — формы и UI.
 - META-INF/context.xml — настройка persistent sessions в Tomcat.
-

1.19 15. Известные технические замечания (честно, как на хорошей защите)

1. DbConnectionFactory после Hibernate-миграции стал legacy и может быть удален.
 2. Main.java не участвует в web flow.
 3. В context.xml путь /tmp/... не кроссплатформенный (для Windows лучше \${catalina.base}/temp/...).
 4. При hbm2ddl=update Hibernate может не покрывать сложные миграции; в production лучше Flyway/Liquibase.
-

1.20 16. Короткий «спич» на 60–90 секунд

«Это Java web-приложение на Servlet/JSP с авторизацией и файловым менеджером. Пользователь регистрируется, логинится, и после этого работает только внутри своего home-каталога. Доступ к файлам защищен от traversal через canonical-path проверку. На стадии added MySQL DB я перенес хранение пользователей из файлов в MySQL и добавил bcrypt-хеширование паролей. В актуальной версии я заменил JDBC-реализацию persistence-слоя на Hibernate: добавил UserEntity, HibernateUtil, и переписал UserRepository на Session/Transaction/HQL, при этом внешний контракт сервлетов и URL не изменился. То есть сделал безопасный рефакторинг инфраструктурного слоя без ломки бизнес-логики и интерфейса.»

1.21 17. Приложение А: команды для диагностики

1.21.1 17.1. Проверка MySQL

```
mysql -u root -p --root -e "SHOW DATABASES;"
mysql -u root -p --root -e "USE explorer; SHOW TABLES; SELECT COUNT(*) FROM users;"
```

1.21.2 17.2. Проверка Tomcat и порта

```
lsof -nP -iTCP:8080 -sTCP:LISTEN
```

1.21.3 17.3. Быстрый smoke test

```
curl -i http://127.0.0.1:8080/explorer/
curl -i http://127.0.0.1:8080/explorer/login
```

1.22 18. Приложение В: каркас устных ответов «почему так»

- Почему не Spring: учебная цель — показать низкоуровневый web lifecycle.
 - Почему bcrypt: защита паролей с солью и адаптивной сложностью.
 - Почему ORM: меньше boilerplate, entity-centric код, проще поддержка модели.
 - Почему сохранил API сервлетов: минимальный риск при миграции persistence.
 - Почему session-based auth: для классического server-rendered приложения это простая и понятная схема.
-

1.23 19. Приложение С: контрольный список перед защитой

1. Приложение открывается по `http://localhost:8080/explorer`.
 2. Регистрация работает.
 3. Логин работает.
 4. Выход работает.
 5. В DBeaver видно запись в `users` с `bcrypt hash`.
 6. Попытка доступа к чужому/внешнему пути возвращает ошибку доступа.
 7. Можешь словами объяснить разницу:
 - `added auth` (файлы)
 - `added MySQL DB (JDBC)`
 - текущая версия (Hibernate)
-

1.24 20. Итог

Проект демонстрирует последовательную эволюцию:

- от простого файлового MVP,
- к централизованной БД и безопасному хранению паролей,
- к ORM-подходу с Hibernate.

Это хороший учебный пример того, как менять внутреннюю реализацию слоя данных, сохраняя внешнее поведение приложения стабильным.

1.25 21. Servlet контейнер: глубокий разбор lifecycle

1.25.1 21.1. Что делает Tomcat на старте

1. Загружает `webapp` из `explorer.war`.
2. Инициализирует классы сервлетов по мере первого обращения (`lazy init`, если не задан `load-on-startup`).
3. Для каждого HTTP-запроса создаёт/использует `HttpServletRequest` и `HttpServletResponse`.
4. Вызывает нужный метод (`doGet`, `doPost`) у `singleton`-экземпляра сервлета.

Важный момент для защиты: экземпляр сервлета обычно один, поэтому поля сервлета разделяются между потоками. В проекте состояние запроса хранится в локальных переменных методов — это правильно.

1.25.2 21.2. Forward vs Redirect (частый вопрос)

- `forward(RequestDispatcher.forward)` — серверная переадресация без нового HTTP-запроса; URL в браузере не меняется.

- `sendRedirect` — клиентский редирект (HTTP 302); браузер делает новый запрос.

Где используется:

- Формы ошибок (`login.jsp`, `register.jsp`) — `forward`, чтобы показать `validation/error` в том же запросе.
- Переходы после успешных действий (`/files`, `/login`) — `redirect`.

1.25.3 21.3. Session lifecycle в этом проекте

- Логин/регистрация: `req.getSession(true) + setAttribute("userLogin", ...)`.
- Выход: `session.invalidate()`.
- Protected endpoints проверяют `req.getSession(false)` и наличие `userLogin`.

1.25.4 21.4. Потокобезопасность

- `HttpSession` объект в контейнере потокобезопасен на уровне API, но бизнес-операции нужно проектировать аккуратно.
 - В проекте нет изменяемого `shared-state` в сервлетах — это плюс.
-

1.26 22. Hibernate internals: что могут спросить глубже

1.26.1 22.1. Entity states

Состояния сущности:

1. `transient` — объект создан `new`, ORM о нем еще не знает.
2. `persistent/managed` — объект связан с текущей `Session`.
3. `detached` — объект был `managed`, но `session` закрыта.
4. `removed` — объект помечен к удалению.

В проекте `new UserEntity(...) -> session.persist(...)` переводит объект в `managed-state` до `commit`.

1.26.2 22.2. Flush и Commit

- `flush` — синхронизация изменений из `persistence context` в SQL.
- `commit` — завершение транзакции в БД.

В текущем коде явный `flush` не вызывается, Hibernate делает `flush` сам перед `commit`.

1.26.3 22.3. Почему SessionFactory static final

SessionFactory дорогой объект: строит метамодель сущностей, диалект, SQL стратегии. Делать его на каждый запрос нельзя.

1.26.4 22.4. Почему Session на операцию

Session не thread-safe. В проекте она открывается в try-with-resources на каждую операцию репозитория — корректно для учебного уровня.

1.26.5 22.5. Что делает hbm2ddl.auto=update

На старте Hibernate сравнивает mapping с текущей схемой и пытается добавить/скорректировать структуру без полного drop. Это удобно для dev, но в production чаще используют Flyway/Liquibase.

1.26.6 22.6. Почему в логе предупреждение про dialect

В Hibernate 6 часть dialect определяется автоматически по JDBC metadata, поэтому явный MySQLDialect иногда не обязателен. Это warning, не ошибка.

1.27 23. Полная матрица эволюции (3 стадии)

Аспект	added auth	added MySQL DB	Актуальная Hibernate версия
Хранение users	файлы .properties	MySQL таблица users	MySQL + ORM mapping
Пароль	plaintext	bcrypt	bcrypt
Persistence API	file IO (Properties)	JDBC	Hibernate/JPA annotations
Инфраструктура	без БД	DbConnectionFactory	HibernateUtil + SessionFactory
Создание схемы	не нужно	SQL CREATE TABLE IF NOT EXISTS	hbm2ddl=update
Запрос логина	чтение файла	SQL SELECT password FROM users WHERE login=?	HQL select u from UserEntity u where u.login=:login
Обработка duplicate	проверка наличия файла	ловля SQL unique exception	ловля constraint exception в ORM
Риск утечки пароля	высокий	низкий	низкий

Аспект	added auth	added MySQL DB	Актуальная Hibernate версия
Масштабируемость	низкая	средняя	средняя+

1.28 24. Разбор ключевых строк по файлам (для «покажи в коде»)

1.28.1 24.1. AuthService

- BASE_DIR и HOMES_DIR задают файловую корневую область пользователя.
- register: валидация + хеш + вызов persistence.
- ensureDatabase: единая точка bootstrap БД.
- resolveInsideHome/isInsideHome: фундаментальная защита пути.

1.28.2 24.2. UserRepository

- ensureSchema: инициализация ORM.
- createUser: транзакция + persist + rollback.
- authenticate: параметризованный HQL.
- isDuplicateConstraint: traversal цепочки exceptions.

1.28.3 24.3. HibernateUtil

- readConfig: приоритет System.getProperty > env > default.
- SESSION_FACTORY static final: lazy static init при первом обращении.
- configuration.addAnnotatedClass(UserEntity.class) — регистрация mapping.

1.28.4 24.4. Servlet-слой

- В каждом protected endpoint есть gate userLogin из session.
- FilesServlet и DownloadServlet не доверяют path с клиента и всегда делают home-проверку.

1.29 25. Каверзные вопросы и как на них отвечать

1.29.1 25.1. «Почему не оставили JDBC, если он проще и прозрачнее?»

JDBC проще в маленьком CRUD, но растёт boilerplate и цена поддержки при усложнении модели. Hibernate уменьшает ручной SQL-код и даёт объектную модель, что важно для масштабирования доменной логики.

1.29.2 25.2. «Что будет, если Hibernate не сможет поднять SessionFactory?»

HibernateUtil бросит IllegalStateException; UserRepository.ensureSchema оборачивает в SQLException; AuthService оборачивает в IOException; в итоге запрос завершится 500, что ожидаемо для инфраструктурного сбоя.

1.29.3 25.3. «Почему не используете @Transactional?»

Потому что проект без Spring/Jakarta EE DI-контейнера. Транзакции открываются вручную через Session API.

1.29.4 25.4. «Не слишком ли рано вы делаете ensureDatabase() на login/register?»

Для учебного проекта это практичный fail-fast: если БД недоступна, пользователь получает корректный сигнал сразу при auth-операции.

1.29.5 25.5. «Почему login/email уникальны и что будет при конфликте?»

Это бизнес-инвариант. Конфликт ловится и преобразуется в понятное сообщение «Пользователь с таким логином или email уже существует».

1.29.6 25.6. «Возможна ли SQL/HQL injection?»

На текущих запросах нет, потому что используется параметр :login и ORM binding.

1.29.7 25.7. «Почему не сделали слой DAO интерфейс + impl?»

Для объема учебного проекта это избыточно. Один репозиторий даёт достаточную тестируемость и прозрачность.

1.29.8 25.8. «Как проверяется принадлежность файла пользователю?»

Сравнение canonical path цели с canonical path home (equals или startsWith(home + separator)).

1.29.9 25.9. «Почему в проекте есть Main.java?»

Это IDE-шаблон, в web-runtime не используется. Можно удалить без влияния.

1.29.10 25.10. «Почему в проекте остался DbConnectionFactory?»

После ORM-миграции это legacy. Оставлен временно, но может быть удален.

1.30 26. Что лучше улучшить после защиты (если спросят roadmap)

1. Удалить legacy классы (DbConnectionFactory, Main).
 2. Привести context.xml к кроссплатформенному пути (\${catalina.base}/temp/...).
 3. Добавить AuthFilter для централизованной защиты маршрутов.
 4. Добавить CSRF token.
 5. Вынести конфиг БД в profile-specific файл/секреты.
 6. Добавить integration tests для register/login и path-guard.
 7. Добавить pagination/limit для больших директорий.
-

1.31 27. Мини-шпаргалка (одной страницей)

- Архитектура: Servlet -> AuthService -> UserRepository -> Hibernate -> MySQL.
 - Ключевая защита: bcrypt + resolveInsideHome.
 - Ключевой evolution:
 - файлы -> JDBC/MySQL (added MySQL DB) -> Hibernate.
 - Где транзакция: UserRepository.createUser.
 - Где session login state: HttpSession.userLogin.
 - Почему ORM: меньше boilerplate, entity-centric код.
 - Что показать в демо: register -> files -> DBever row -> logout -> login.
-

1.32 28. Приложение D: разница по файлам (конкретно)

1.32.1 28.1. Что добавил added MySQL DB

- src/main/java/org/example/DbConnectionFactory.java
- src/main/java/org/example/PasswordService.java
- src/main/java/org/example/UserRepository.java

Изменил:

- src/main/java/org/example/AuthService.java
- pom.xml

1.32.2 28.2. Что добавила текущая Hibernate-миграция (working tree)

Добавлены:

- src/main/java/org/example/HibernateUtil.java
- src/main/java/org/example/UserEntity.java

Изменены:

- src/main/java/org/example/UserRepository.java
- pom.xml

1.32.3 28.3. Что внешне не поменялось

- URL endpoints
- JSP формы
- Логика session-based auth
- Логика home-dir isolation

1.33 29. Приложение E: сверхкраткий рассказ по commit-истории

1. init — стартовый file explorer без auth.
 2. added auth — регистрация/логин, сессии, отдельные home-каталоги.
 3. added MySQL DB — перенос пользователей в MySQL + bcrypt.
 4. Текущая версия — перенос persistence-реализации с JDBC на Hibernate.
-

1.34 30. Финальный вывод для защиты

Если преподаватель спрашивает «что здесь главное инженерно», отвечай так:

- «Я последовательно улучшал архитектуру, не ломая внешний контракт системы».
- «Сначала сделал auth и изоляцию данных пользователя, потом вынес хранение учеток в БД и защитил пароли, затем сделал инфраструктурную миграцию JDBC -> Hibernate».
- «Это демонстрирует, что я понимаю не только как написать фичу, но и как эволюционно рефакторить систему по слоям».